

Docker 入门学习笔记

镜像 · 容器 · 端口 · 挂载 · 网络 · Compose · Dockerfile

```
$ docker compose up -d
$ docker logs -f app
$ docker compose down
```

Docker 入门到能部署项目

一篇从心智模型开始的 Docker 学习教程

Docker 入门到能部署项目：一篇真正能复习的学习笔记

最近看了一套 Docker 视频课，课程顺序很舒服：先讲 Docker 是什么，再讲镜像、容器、docker run、挂载、网络、Docker Compose、Dockerfile，最后讲日志、清理和销毁实例。

我看完以后最大的感受是：Docker 不是一堆命令，也不是“服务器上多装一个软件”。它真正改变的是部署方式。以前部署项目靠手工记步骤，现在可以把环境、依赖、启动命令、端口、网络写进文件里，让部署过程变得可复制。

这篇文章按我自己的理解重新整理，不追求把所有命令都列完，而是帮自己建立一套 Docker 的心智模型。

1. 先建立一个整体画面

学 Docker 最容易晕，是因为它同时出现了很多词：

- 镜像
- 容器

- 仓库
- 端口映射
- 数据卷
- 网络
- Dockerfile
- Compose

先不要急着背命令，先看它们之间的关系：

Dockerfile 负责制作镜像
镜像 image 是应用运行环境的模板
容器 container 是镜像运行起来后的实例
仓库 registry 用来存放和分发镜像
端口映射让外部访问容器里的服务
挂载让容器之外保存数据
网络让多个容器互相访问
Compose 把一组容器的运行规则写进一个 yml 文件

如果把部署项目看成做饭，Dockerfile 像菜谱，镜像像预制好的食材包，容器是正在锅里运行的那份菜，Compose 是一桌菜的上菜清单：哪个菜要先做，用什么锅，端口放哪，调料放哪。

真正部署时，我们通常只关心两件事：

怎么把项目做成镜像
怎么把镜像稳定运行成容器

Dockerfile 解决第一件事，Docker Compose 解决第二件事。

2. Docker 解决的不是“安装软件”，而是“环境一致”

以前部署一个网站，可能是这样的：

```
apt install nodejs
npm install -g pnpm
git clone 项目
pnpm install
pnpm build
pnpm start
```

这条路能跑，但问题很多：

- 服务器 A 的 Node 是 18，服务器 B 的 Node 是 22
- 本地 pnpm 版本和服务器不一致
- 某个依赖需要系统库，另一台机器没有
- 环境变量忘记配置
- 端口被占用

- 项目挂了以后没人自动拉起
- 换服务器时又要重新做一遍

Docker 的思路是：不要让服务器承受这么多不确定性。服务器只需要安装 Docker，项目需要的运行环境都写进镜像。

这就是 Docker 最核心的价值：

把“我手动配置过的环境”，变成“可以重复构建的环境”。

所以学 Docker 时，不要只问“这个命令怎么用”，更要问：

这个命令是在解决部署过程里的哪个不确定性？

3. 镜像 image：应用环境的模板

镜像是在 Docker 里最基础的概念。它不是正在运行的服务，而是一个只读模板。

比如：

```
nginx:latest
redis:7
mysql:8
node:22-alpine
ubuntu:24.04
```

这些都是镜像。

常用命令：

```
docker images
docker pull nginx
docker rmi nginx
```

`docker pull nginx` 是从镜像仓库下载 nginx 镜像。下载以后，本机就有了一个 nginx 模板。

镜像和容器的关系可以这样理解：

```
镜像 是 模板
容器 是 运行实例

同一个镜像可以启动多个容器
删除容器不会删除镜像
删除镜像前通常要先删除依赖它的容器
```

如果你学过编程，可以类比成：

```
类 class -> 对象 object
镜像 image -> 容器 container
```

不过这个类比不用太较真，只是帮助理解。

4. 容器 container：真正跑起来的应用

镜像不会自己运行。镜像运行起来以后，才叫容器。

最简单的启动方式：

```
docker run nginx
```

这条命令会启动 nginx，但它会占住当前终端。更常用的是后台运行：

```
docker run -d --name my-nginx -p 8080:80 nginx
```

拆开看：

```
docker run：根据镜像创建并启动容器  
-d：后台运行  
--name my-nginx：给容器取名  
-p 8080:80：端口映射  
nginx：使用 nginx 镜像
```

查看正在运行的容器：

```
docker ps
```

查看所有容器，包括已经停止的：

```
docker ps -a
```

停止容器：

```
docker stop my-nginx
```

启动已经停止的容器：

```
docker start my-nginx
```

删除容器：

```
docker rm my-nginx
```

如果容器还在运行，需要先停掉，或者强制删除：

```
docker rm -f my-nginx
```

平时排查时，我最常用的是：

```
docker ps -a  
docker logs -f 容器名
```

一个看状态，一个看原因。

5. 端口映射：外面访问的是宿主机端口

端口映射是初学 Docker 最容易写反的地方。

格式是：

宿主机端口:容器端口

比如：

```
docker run -d -p 8080:80 nginx
```

意思是：

访问服务器 8080 端口
转发到容器里的 80 端口

所以你在浏览器访问：

http://服务器IP:8080

实际请求会进入 nginx 容器里的 80 端口。

判断左右的方法：

左边：别人访问你的服务器时用的端口
右边：应用在容器内部监听的端口

如果你把它写成：

```
-p 80:8080
```

意思就完全变了。它表示服务器 80 端口转发到容器 8080 端口。假如容器里 nginx 监听的是 80，那就访问不到。

排查端口时可以这样看：

```
docker ps
ss -tunlp | grep 8080
curl http://127.0.0.1:8080
```

docker ps 能看到端口映射关系，ss 能看宿主机端口是否被监听，curl 能直接在服务器内部测试服务是否通。

6. 日志：别猜，先看 logs

Docker 排障第一条原则：先看日志。

单容器：

```
docker logs 容器名
docker logs -f 容器名
```

```
docker logs --tail=100 容器名
```

Compose :

```
docker compose logs
docker compose logs -f
docker compose logs -f 服务名
docker compose logs --tail=100 服务名
```

日志能帮你快速判断问题类型：

- 依赖下载失败
- 启动命令写错
- 端口冲突
- 环境变量缺失
- 数据库连不上
- 权限不足
- 配置文件路径错误
- 应用代码报错

很多时候不是 Docker 有问题，而是应用自己启动失败。Docker 只是把错误打印出来。

我现在习惯按这个顺序排查：

```
docker compose ps
docker compose logs -f
docker inspect 容器名
```

先看容器是不是反复重启，再看日志，再看详细配置。

7. 挂载：容器可以删，数据不能丢

容器本身不适合当数据保险箱。容器删除后，容器内部临时写入的数据可能就没了。

所以需要挂载。

Docker 常见挂载方式有两类：

- bind mount：宿主机目录挂到容器里
- volume：Docker 管理的数据卷

目录挂载示例：

```
docker run -d \
  --name my-nginx \
  -p 8080:80 \
  -v /opt/nginx/html:/usr/share/nginx/html \
  nginx
```

意思是：

- 宿主机目录 /opt/nginx/html
- 挂载到容器目录 /usr/share/nginx/html

容器读写 /usr/share/nginx/html，本质上就是读写宿主机的 /opt/nginx/html。

数据卷示例：

```
docker volume create mysql-data
```

```
docker run -d \  
  --name mysql \  
  -v mysql-data:/var/lib/mysql \  
  mysql:8
```

两种方式怎么选？

你想直接在宿主机看到文件：用 bind mount

你只想让 Docker 管数据：用 volume

配置文件、上传目录：常用 bind mount

数据库数据：常用 volume

权限问题也经常出现在挂载这里。比如容器里应用要写 /app/uploads，但宿主机对应目录没有写权限，就会报 Permission denied。

排查：

```
ls -ld /宿主机目录  
id  
docker compose logs -f
```

不要只看容器内部路径，也要看宿主机目录权限。

8. 网络：容器里的 127.0.0.1 不是宿主机

Docker 网络是另一个容易踩坑的地方。

假设你有两个容器：

```
app  
redis
```

如果 app 容器里写：

```
127.0.0.1:6379
```

它访问的是 app 容器自己，不是 redis 容器，也不是宿主机。

正确做法是让两个容器在同一个 Docker 网络里，然后用服务名访问：

```
redis:6379
```

手动创建网络：

```
docker network create app-net
```

启动 Redis :

```
docker run -d --name redis --network app-net redis:7
```

启动应用 :

```
docker run -d --name app --network app-net my-app
```

同一个网络里，容器可以通过容器名互相访问。

Docker Compose 会自动给项目创建默认网络，所以 Compose 里通常直接用服务名访问：

```
services:
  app:
    image: my-app
    environment:
      REDIS_URL: redis://redis:6379

  redis:
    image: redis:7
```

这里 app 访问的不是 127.0.0.1:6379，而是 redis:6379。

这个点记住以后，很多“容器里连不上数据库”的问题就能自己判断了。

9. Docker Compose：把运行方式写成文件

docker run 适合学习单个容器，但真实项目里命令会越来越长：

- 端口
- 环境变量
- 挂载目录
- 网络
- 重启策略
- 多个服务
- 服务依赖

继续手写命令会很难维护。

Docker Compose 的作用就是把这些参数写进 docker-compose.yml。

一个最小例子：

```
services:
  web:
    image: nginx
    ports:
      - "8080:80"
    restart: unless-stopped
```


启动：

```
docker compose up -d
```

停止并删除：

```
docker compose down
```

查看：

```
docker compose ps
docker compose logs -f
```

Compose 文件可以理解成：

- 我要哪些服务
- 每个服务用什么镜像
- 映射哪些端口
- 设置哪些环境变量
- 挂载哪些目录
- 异常退出后要不要重启

这就是“部署配置代码化”。

10. Compose 常见字段怎么读

看一个稍完整的例子：

```
services:
  app:
    image: my-app:latest
    restart: unless-stopped
    ports:
      - "3000:3000"
    environment:
      NODE_ENV: production
      REDIS_URL: redis://redis:6379
    volumes:
      - ./uploads:/app/uploads
    depends_on:
      - redis

  redis:
    image: redis:7
    restart: unless-stopped
    volumes:
      - redis-data:/data

volumes:
  redis-data:
```

逐个解释：

services：服务列表
app：服务名

image : 使用的镜像
restart : 重启策略
ports : 端口映射
environment : 环境变量
volumes : 挂载目录或数据卷
depends_on : 启动顺序依赖
volumes 顶层字段 : 声明 Docker 管理的数据卷

depends_on 只保证启动顺序，不保证 Redis 已经完全可用。所以生产项目里，应用最好自己有重试逻辑。

常见重启策略：

no : 不自动重启
always : 总是重启
unless-stopped : 除非手动停止，否则自动重启
on-failure : 失败时重启

个人项目和普通 Web 服务，我通常会用：

```
restart: unless-stopped
```

它比较符合直觉。

11. Dockerfile：把项目做成镜像

Compose 管“怎么运行”，Dockerfile 管“怎么构建镜像”。

一个 Node 项目的简化 Dockerfile：

```
FROM node:22-alpine

WORKDIR /app

COPY package.json pnpm-lock.yaml ./
RUN npm install -g pnpm && pnpm install --frozen-lockfile

COPY . .
RUN pnpm build

EXPOSE 3000
CMD ["pnpm", "start"]
```

常见指令：

FROM : 基于哪个基础镜像
WORKDIR : 设置工作目录
COPY : 复制文件
RUN : 构建镜像时执行命令
EXPOSE : 声明容器内部端口
CMD : 容器启动时执行命令

最容易混的是 RUN 和 CMD。

RUN 在构建镜像时执行
CMD 在容器启动时执行

安装依赖、编译项目，放在 RUN。

启动 Web 服务，放在 CMD。

12. Dockerfile 顺序会影响构建速度

Docker 镜像是分层的。Dockerfile 每一步都会形成一层缓存。

所以 Node 项目通常这样写：

```
COPY package.json pnpm-lock.yaml ./
RUN pnpm install --frozen-lockfile

COPY . .
RUN pnpm build
```

不要一上来就：

```
COPY . .
RUN pnpm install
```

原因是业务代码经常改，但 package.json 和锁文件不一定经常改。

如果先复制锁文件再安装依赖，Docker 可以缓存依赖安装层。下次只改业务代码时，就不用重新下载依赖，构建会快很多。

这就是 Dockerfile 的一个重要经验：

```
变化少的步骤放前面
变化多的步骤放后面
```

13. 多阶段构建：构建环境和运行环境分开

真实项目里，经常不希望把所有构建工具都带到最终镜像里。

比如构建时需要：

```
pnpm
TypeScript
构建缓存
源码
开发依赖
```

但运行时可能只需要：

```
构建产物
少量 node_modules
启动脚本
```

多阶段构建就是为了解决这个问题。

示例：

```
FROM node:22-alpine AS deps
WORKDIR /app
COPY package.json pnpm-lock.yaml ./
RUN npm install -g pnpm && pnpm install --frozen-lockfile
```

```
FROM node:22-alpine AS builder
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY . .
RUN pnpm build
```

```
FROM node:22-alpine AS runner
WORKDIR /app
ENV NODE_ENV=production
COPY --from=builder /app/dist ./dist
CMD ["node", "dist/index.js"]
```

好处：

- 最终镜像更小
- 运行环境更干净
- 少带一些不需要的构建文件
- 部署时更稳定

Next.js 项目常见的 standalone 输出，也是类似思路：只把运行需要的文件拎出来。

14. .dockerignore：别把垃圾塞进镜像

.dockerignore 很容易被忽略，但它很重要。

如果没有 .dockerignore，构建时可能会把这些东西也传进 Docker：

- node_modules
- .git
- .next
- 日志文件
- 本地 .env
- 临时文件

这会导致构建慢、镜像大，甚至把敏感文件打进去。

常见写法：

```
node_modules
.git
.next
dist
```

```
coverage
.env
.env.*
*.log
```

它的作用有点像 .gitignore，但服务的是 Docker 构建上下文。

15. 环境变量：配置不要写死在代码里

应用运行经常需要配置：

- 数据库地址
- Redis 地址
- API Key
- 运行环境
- 站点域名

Docker 里可以通过 environment 传入：

```
services:
  app:
    image: my-app
    environment:
      NODE_ENV: production
      API_BASE_URL: https://api.example.com
```

也可以用 .env 文件配合 Compose：

```
services:
  app:
    image: my-app
    environment:
      NODE_ENV: ${NODE_ENV}
```

不过要注意：前端项目里的 NEXT_PUBLIC_*、VITE_* 这类变量通常会在构建时写进静态资源。也就是说，改了这些变量后，可能不是重启容器就行，而是要重新 build。

这个点在前端 Docker 部署里很常见。

16. 从 docker run 到 Compose 的迁移思路

假设原来有一条命令：

```
docker run -d \
  --name app \
  -p 3000:3000 \
  -e NODE_ENV=production \
  -v ./uploads:/app/uploads \
  --restart unless-stopped \
  my-app:latest
```

可以翻译成 Compose :

```
services:
  app:
    image: my-app:latest
    restart: unless-stopped
    ports:
      - "3000:3000"
    environment:
      NODE_ENV: production
    volumes:
      - ./uploads:/app/uploads
```

对应关系 :

```
--name app -> 服务名 app
-p -> ports
-e -> environment
-v -> volumes
--restart -> restart
镜像名 -> image
```

所以 Compose 并不是新东西，它只是把长命令变成结构化配置。

17. 常用命令速查

镜像 :

```
docker images
docker pull nginx
docker rmi nginx
docker build -t my-app:latest .
```

容器 :

```
docker ps
docker ps -a
docker start 容器名
docker stop 容器名
docker restart 容器名
docker rm 容器名
docker logs -f 容器名
docker exec -it 容器名 sh
```

网络 :

```
docker network ls
docker network create app-net
docker network inspect app-net
```

数据卷 :

```
docker volume ls
docker volume create data
```

```
docker volume inspect data
```

Compose :

```
docker compose up -d
docker compose up -d --build
docker compose ps
docker compose logs -f
docker compose restart
docker compose down
```

清理 :

```
docker system df
docker image prune
docker container prune
docker builder prune
docker system prune
```

清理命令要谨慎，尤其是 `docker system prune`。它会清理未使用的镜像、容器、网络 and 缓存。执行前先看：

```
docker system df
```

18. 排障时按层次查

Docker 出问题时，别一上来就重装。

我会按这个顺序查：

1. 容器有没有运行
2. 日志有没有报错
3. 端口有没有映射
4. 宿主机端口有没有监听
5. 容器内部服务有没有启动
6. 环境变量是否正确
7. 挂载目录权限是否正确
8. 容器网络是否能互通
9. Nginx 或反向代理是否配置正确

对应命令：

```
docker compose ps
docker compose logs -f
docker ps
ss -tunlp
curl http://127.0.0.1:端口
docker exec -it 容器名 sh
docker inspect 容器名
```

如果是 Nginx 502，我会这样查：

```
nginx -t
tail -f /var/log/nginx/error.log
```

```
curl http://127.0.0.1:后端端口
docker compose logs -f
```

判断逻辑：

```
curl 后端端口不通：先修 Docker 或应用
curl 后端端口通，域名不通：再修 Nginx
Nginx 配置没通过：先修 nginx.conf
容器反复重启：先看 docker compose logs
```

19. 学 Docker 最该记住的几个坑

第一，容器里的 127.0.0.1 是容器自己，不是宿主机。

第二，端口映射左边是宿主机，右边是容器。

第三，容器删了，没挂载的数据可能就没了。

第四，docker compose down 会删容器和网络，但一般不会删命名 volume。带 -v 才会连 volume 一起删。

第五，前端构建变量可能需要重新 build，不是 restart 就能生效。

第六，depends_on 不等于服务已经完全可用。

第七，不要随便改 /var/run/docker.sock 权限，权限问题优先用用户组或 root 解决。

第八，构建慢时先看 Dockerfile 顺序和 .dockerignore。

第九，日志永远比猜测可靠。

20. 一个完整的小练习

如果要练手，可以用 nginx 做一个最小项目。

创建目录：

```
mkdir -p /opt/docker-demo/html
cd /opt/docker-demo
```

写一个页面：

```
echo '<h1>Hello Docker</h1>' > html/index.html
```

写 docker-compose.yml：

```
services:
  web:
    image: nginx
    restart: unless-stopped
    ports:
      - "8080:80"
    volumes:
```



```
- ./html:/usr/share/nginx/html
```

启动：

```
docker compose up -d
```

访问：

```
http://服务器IP:8080
```

修改 `html/index.html` 后刷新页面，你会发现内容变了。这就是挂载的效果。

停止并清理：

```
docker compose down
```

这个练习虽然简单，但它串起了：

- Compose
- 镜像
- 容器
- 端口映射
- 目录挂载
- 日志
- 销毁实例

很适合刚学完时跑一遍。

21. 我现在怎么理解 Docker

以前我以为 Docker 是一种“更高级的安装方式”。现在我更愿意把它理解成：

Docker 是把部署过程写成文件的工具。

Dockerfile 描述环境怎么做出来。

Compose 描述服务怎么跑起来。

镜像让环境可以分发。

容器让应用隔离运行。

挂载让数据留在容器外面。

网络让服务之间互相找到。

日志让排障有入口。

所以 Docker 真正重要的不是某个命令，而是这套工程习惯：

- 不要依赖手工配置
- 不要让环境停留在口头说明里

不要把数据只放在容器内部
不要靠猜测排错
把能写成文件的东西写成文件
把能重复执行的流程固定下来

学到这里，Docker 就不只是“我会启动一个 nginx 容器”，而是“我能把一个项目整理成可部署、可迁移、可排查的服务”。

这才是 Docker 对个人项目、服务器部署、Web 应用和后续学习 Kubernetes 真正有用的地方。